
Software Requirements Specification

for

QR-BASED FILE SHARING SYSTEM

Version 1.0

Prepared by

GroupName:A

MD AAQUIB
ARGHADEEP DEBNATH

CS-K
CS-K

2023239993.md@ug.sharda.ac.in
[2023452365 .arghadeep@ug.sharda.ac.in](mailto:2023452365.arghadeep@ug.sharda.ac.in)

Instructor: Prof. Dr. Shajee Mohan B.S.

Course: Software Engineering and
Testing Methodologies Lab

22-August-2025

830-1984 - IEEE Guide for Software
Requirements

Contents

Introduction

- 1.1 Purpose
- 1.2 Scope
- 1.3 Definitions, Acronyms, and Abbreviations
- 1.4 References
- 1.5 Overview

Overall Description

- 2.1 Product Perspective
- 2.2 Product Functions
- 2.3 User Classes and Characteristics
- 2.4 Operating Environment
- 2.5 Design and Implementation Constraints
- 2.6 User Documentation
- 2.7 Assumptions and Dependencies

Specific Requirements

- 3.1 Functional Requirements
- 3.2 External Interface Requirements
- 3.3 System Features
- 3.4 Non-Functional Requirements

System Models and Diagrams

- 4.1 Use Case Diagram
- 4.2 System Architecture Diagram
- 4.3 Data Flow Diagram (DFD)

Performance Requirements

Security Requirements

Quality Attributes

Other Requirements

Appendices

Version	Primary Author(s)	Description of Version	Date Completed
1.0	MD AAQUIB	This is Version 1.0 of the QR-Based File Sharing System, providing secure peer-to-peer file transfer via QR code authentication and Wi-Fi Direct. Future versions may include cloud integration and transfer history features.	22/08/25
1.0	AGARHDEEP DEBNATH	This is Version 1.0 of the QR-Based File Sharing System, providing secure peer-to-peer file transfer via QR code authentication and Wi-Fi Direct. Future versions may include cloud integration and transfer history features.	22/08/25

1 Introduction

Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a comprehensive description of the **QR-Based File Sharing System**. The document details the functional and non-functional requirements of the system, user classes, constraints, and overall product perspective. It serves as a reference for developers, testers, project managers, and stakeholders involved in the development and deployment of the system.

The primary goal of the system is to allow users to transfer files securely and quickly between devices using **QR codes for pairing** instead of manual entry of connection details. This approach improves usability, reduces errors, and ensures a smooth user experience.

Product Scope

The **QR-Based File Sharing System** is a mobile application that enables users to send and receive files (documents, images, videos, etc.) by scanning a QR code generated by the sender or receiver. The system supports offline transfer via **Wi-Fi Direct** or local hotspot, eliminating the need for internet connectivity.

Key Features:

- **QR Code Pairing:** Establishes a secure connection between devices quickly.
- **Fast Transfer:** Ensures high-speed file transfer for small and large files.
- **Security:** Implements AES-128 encryption for secure data transmission.
- **Cross-Platform Support (Future Scope):** Potential expansion to iOS and Web.

Definitions , Acronyms and Abbreviations

- **QR Code:** Quick Response code used for storing session details for pairing.
- **AES:** Advanced Encryption Standard, used for securing file transfer.
- **Wi-Fi Direct:** A technology allowing direct wireless connections between devices without a router.
- **App:** Mobile application.

Document Conventions

This SRS is organized by numbered sections. Functional requirements are labeled “F1, F2, ...” and non-functional as “N1, N2, ...”. The word “shall” denotes a requirement. Bullet lists summarize grouped information for clarity. Diagrams (Figs. 1, 2) illustrate system context and use cases.

References and Acknowledgments

- IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications.
- Android Developer Guide – <https://developer.android.com>
- Research Paper: *Secure File Transfer via QR Code and Wi-Fi Direct* – Springer, 2023.
- AES Encryption Standard – NIST, 2001.

2 Overall Description

Product Overview

The **QR-Based File Sharing System** is a mobile application designed to provide a **fast, secure, and user-friendly** way of transferring files between devices. Instead of relying on internet connectivity or lengthy pairing processes, the system uses **QR codes** to establish connections quickly.

Key Highlights

- Eliminates the need for manual device pairing or sharing of IP addresses.
- Supports offline transfer using **Wi-Fi Direct or mobile hotspot**.
- Ensures security with **AES-128 encryption** and temporary session tokens.
- Designed to work on **Android (minimum version 8.0)** with potential future expansion to **iOS and web platforms**.

Product Functionality

1. QR Code-Based Device Pairing

- **Generate QR Code** – The sender's device creates a QR code containing connection details (IP address, port number, and encryption key).
- **Scan QR Code** – The receiver scans the QR code to establish a secure connection instantly.
- **Temporary Session** – Connection remains active only for the duration of the transfer, ensuring privacy.

2. File Transfer Module

- **Send Files** – Users can select files (documents, images, videos, audio) from their device.
- **Receive Files** – Users can receive files after scanning the sender's QR code.
- **Progress Indicator** – Displays real-time transfer speed, percentage completed, and estimated time.
- **Resume Support** (Future Enhancement) – Paused or interrupted transfers can be resumed.

3. Security and Encryption

- **AES-128 Encryption** – All files are encrypted before transfer to ensure confidentiality.
 - **Checksum Verification** – Confirms file integrity after transfer.
 - **Temporary Session Keys** – Prevent unauthorized access to transfer sessions.
-

4. User Interface (UI) Features

- **Simple Two-Button Dashboard** – “Send” and “Receive” options for quick navigation.
 - **History Log** – (Optional) Displays recent transfers for user reference.
 - **Notifications** – Alerts users upon transfer completion or errors.
-

5. Offline Operation

- **Wi-Fi Direct / Hotspot Transfer** – Eliminates the need for internet connectivity.
- **Automatic Connection Setup** – No manual pairing or password entry required.

Design and Implementation Constraints

1. Platform Constraints

- Developed primarily for **Android (minimum version 8.0 / Oreo)**.
 - QR scanning relies on the **device camera**; performance may vary based on hardware quality.
 - Wi-Fi Direct must be supported by the device hardware for offline file transfers.
-

2. Security Constraints

- AES-128 encryption must be implemented for secure data transmission.
 - Session tokens generated for QR codes must be valid only for a single transfer session.
 - No file should be stored on an external server to avoid data privacy concerns.
-

3. Performance Constraints

- Application must be capable of handling files up to **2 GB** without crashes or lag.
 - QR code generation and scanning should not take more than **2 seconds**.
 - Transfer speed should be optimized to reach up to **20–40 Mbps** depending on device hardware.
-

4. Development Constraints

- Development language: **Java/Kotlin (Android)**.
 - QR code generation and scanning via **ZXing library**.
 - Encryption handled using **Java Cryptography Extension (JCE)**.
-

5. User Experience Constraints

- The UI must remain simple with minimal navigation steps.
- Color scheme must ensure readability in both dark and light modes.

Assumptions and Dependencies

The development and operation of the **QR-Based File Sharing System** are based on several assumptions and dependencies. It is assumed that users possess smartphones with functional cameras and Wi-Fi Direct capability to enable QR code scanning and high-speed file transfer. The application assumes that users will grant necessary permissions for camera access, storage access, and network connectivity. It is also expected that the devices will have sufficient storage capacity for both sending and receiving files without system crashes or data loss.

The system depends on external libraries, such as the ZXing library, for QR code generation and scanning and the Java Cryptography Extension (JCE) for encryption and decryption processes. Additionally, performance depends on the underlying hardware specifications of the devices, such as processor speed, RAM availability, and Wi-Fi module efficiency. Future updates and compatibility may rely on the latest Android APIs and security patches provided by the operating system.

3 Specific Requirements

External Interface Requirements

User Interfaces

The **QR-Based File Sharing System** is designed with a **minimalistic and intuitive UI** to ensure a smooth user experience. The interface will be consistent across all screens and will follow Android design guidelines for better usability.

Hardware Interfaces

The **QR-Based File Sharing System** interacts with various hardware components of a mobile device to perform its operations efficiently. These interfaces ensure smooth scanning, connection setup, and file transfer.

1. Camera Interface

- Required for **QR Code Scanning** to initiate secure connections.
- Must support **autofocus** for clear image capture.
- Works with ZXing library for fast and accurate QR detection.

2. Wireless Network Interface

- Utilizes **Wi-Fi Direct** or mobile hotspot for offline file transfer.
- Requires **802.11n or higher** Wi-Fi capability for high-speed data exchange.
- Bluetooth is not used due to lower transfer speeds.

Software Interfaces

The system shall log operational data and allow export to external databases. It may optionally receive weather updates or maps via an Internet-connected service. If paired with a mobile app, it shall expose a RESTful API for remote commands.

Functional Requirements

3.5 Functional Requirements

The functional requirements define the key operations and behaviors that the **QR-Based File Sharing System** must perform to achieve its purpose.

1. QR Code Generation

- **Description:** The system shall generate a unique QR code containing encrypted session details (IP address, port number, encryption key).
 - **Input:** User selects “Send File” option.
 - **Output:** Display of scannable QR code on the sender’s device.
-

2. QR Code Scanning

- **Description:** The system shall allow the receiver to scan the QR code to establish a secure connection with the sender.
 - **Input:** Camera access and QR code image.
 - **Output:** Connection established for file transfer.
-

3. File Selection

- **Description:** The system shall allow users to browse and select one or multiple files for transfer.
 - **Input:** File selection from internal/external storage.
 - **Output:** Selected files ready for encryption and transfer.
-

4. File Transfer

- **Description:** The system shall securely transfer files between devices using Wi-Fi Direct.
 - **Input:** Encrypted files and active session.
 - **Output:** Successful transfer of files to the receiver.
-

5. Encryption and Security

- **Description:** The system shall encrypt all files using AES-128 before transmission.
 - **Input:** File data before sending.
 - **Output:** Encrypted data sent securely to receiver.
-

6. Transfer Progress and Notification

- **Description:** The system shall display transfer progress, speed, and estimated time remaining.
- **Input:** Transfer data packets.
- **Output:** Real-time progress updates and completion alert.

Use Case Model

The **Use Case Model** for the QR-Based File Sharing System illustrates how different actors interact with the system to perform specific functions.

Actors

1. **Sender** – Initiates file transfer by generating a QR code and selecting files to send.
2. **Receiver** – Scans QR code and receives files securely.
3. **System** – Handles QR code generation, encryption, transfer, and notifications.

Use Cases

1. **Generate QR Code**
 - **Actor:** Sender
 - **Description:** Creates a QR code with encrypted session details for connection.
2. **Scan QR Code**
 - **Actor:** Receiver
 - **Description:** Scans QR code to connect to sender's device.
3. **Select File(s)**
 - **Actor:** Sender
 - **Description:** Selects one or more files from storage for transfer.
4. **Transfer Files**
 - **Actor:** System
 - **Description:** Encrypts files and sends them over Wi-Fi Direct to the receiver.
5. **Display Transfer Progress**
 - **Actor:** System
 - **Description:** Shows real-time progress, speed, and completion status.

Use Case Diagram (Description for Visual)

- **Actors:** Sender, Receiver, System
- **Connections:**
 - Sender → Generate QR Code → System
 - Receiver → Scan QR Code → System
 - Sender → Select Files → System
 - System → Transfer Files → Receiver
 - System → Display Progress → Both Actors

4 Other Non-functional Requirements

Performance Requirements

The **QR-Based File Sharing System** is designed to provide fast, secure, and reliable performance across a wide range of mobile devices. The system must generate and display a QR code within **2 seconds** of initiating the sending process, ensuring minimal delay for the user. File transfer speed should utilize the maximum capability of Wi-Fi Direct, ideally reaching **20–40 Mbps** on compatible devices. The system must handle files up to **2 GB** without performance degradation, crashes, or data corruption. All encryption and decryption processes must be completed in real-time without noticeable lag. Furthermore, the application should maintain a smooth user interface experience, with screen transitions occurring in less than **500 milliseconds** and no freezing during large file transfers. The system must also manage memory efficiently, ensuring that devices with at least **2 GB of RAM** can run the application without performance issues.

Safety and Security Requirements

The **QR-Based File Sharing System** ensures data safety and user privacy through multiple security measures. All files are encrypted using the **AES-128 encryption standard** before transmission to prevent unauthorized access during transfer. QR codes are dynamically generated with session-specific details, making them valid only for a single transfer session to avoid reuse. The system does not store transferred files on any external server, ensuring data remains private between sender and receiver.

Software Quality Attributes

The **QR-Based File Sharing System** must meet high standards of software quality to ensure reliability, usability, and performance. The key quality attributes include:

1. **Reliability** – The system must operate consistently under various conditions, handling file transfers up to 2 GB without errors, data corruption, or unexpected crashes.
2. **Usability** – The interface should be intuitive, requiring minimal steps to perform sending and receiving actions. Clear instructions, icons, and real-time feedback enhance the user experience.
3. **Performance Efficiency** – The system must utilize Wi-Fi Direct to achieve transfer speeds of 20–40 Mbps, ensure QR code generation within 2 seconds, and maintain smooth UI transitions (<500 ms).
4. **Portability** – The application should be compatible with multiple Android versions (minimum Android 8.0) and adaptable for future updates without major redesigns.
5. **Security** – AES-128 encryption, session-based QR codes, and minimal permission requests ensure user data protection and privacy.
6. **Maintainability** – The codebase will follow modular architecture, making future updates, bug fixes, and feature enhancements easier.

5 Other Requirements

In addition to the functional and non-functional aspects, the **QR-Based File Sharing System** must meet several additional requirements to ensure smooth operation, scalability, and future readiness:

1. **Platform Compatibility**
 - The application should work on Android devices with **Android 8.0 (Oreo)** or higher.
 - Support for **cross-device compatibility** (smartphones and tablets) is essential.
2. **Scalability**
 - The system must be capable of handling multiple file types (images, videos, documents, audio files) and should be easily extendable to support larger file sizes in future updates.
3. **Error Handling and Recovery**
 - The system must detect connection failures and provide clear prompts to retry transfers without data corruption.
 - Error logs should be maintained for debugging and improving performance.
4. **Offline Functionality**
 - The system must allow file transfers without internet access by using **Wi-Fi Direct technology** for peer-to-peer communication.
5. **Minimal Resource Usage**
 - CPU and memory utilization should be optimized to prevent battery drain or performance lag, especially during large file transfers.
6. **Future Expandability**
 - The system design should allow easy integration of additional features, such as **multi-file parallel transfers, transfer history management, and cloud backup support** in future versions.

Appendix A–Data Dictionary

Appendix B-Group Log